Facens

Web Design



JS: Sintaxe, Variáveis e Estruturas

Prof. Me. Deivison S. Takatu
Email: deivison.takatu@facens.br

NOTA MÁXIMA NO MEC 5

Datas de Apresentação da AC2

- A partir dos grupos formados na disciplina, será organizado em mais de um dia as apresentações;
- A quantidade máxima de apresentações em um dia deve ser quatro;
- A partir do sorteio das datas, caso seja necessário alteração, avisar com o máximo de antecedência possível.

Javascript

JavaScript é uma linguagem de programação que atua principalmente no lado do cliente (client side), ou seja, é executada diretamente no navegador do usuário. Essa característica permite maior interatividade nas páginas da web, tornando a experiência do usuário mais dinâmica e responsiva.

Além disso, o JavaScript possui diversos usos, como validação de formulários, alteração de estilos, inserção dinâmica de conteúdo e comunicação com o servidor.



Javascript

- O JavaScript pode ser inserido diretamente no corpo do HTML usando a tag <script>, permitindo executar funções simples sem a necessidade de arquivos externos.
- Outra opção é usar arquivos externos, vinculados com o atributo src da tag <script>, prática recomendada em projetos maiores por facilitar organização e manutenção do código.
- Também é possível incluir JavaScript nos atributos de eventos, como onclick e onblur, que tornam a página mais interativa ao responder às ações do usuário.



Javascript

O JavaScript oferece uma ampla variedade de recursos que permitem controlar praticamente todos os aspectos dinâmicos de uma página web. Além dos conceitos básicos abordados em aula, existem inúmeras funcionalidades e APIs especializadas para criar interatividade avançada, manipular o DOM, desenvolver animações, aplicar efeitos visuais, consumir dados de servidores e construir aplicações web modernas e responsivas.

Referência: developer.mozilla.org



Javascript

```
<!-- Corpo da tag SCRIPT -->
<script>
    var teste = "Ola";
</script>

<!-- Atributo src -->
<script src="script.js"></script>

<!-- Atributos de eventos -->
<button onclick="alert('Ola')">Teste</button>
```

Fonte: Elaboração Própria.

Javascript - Variáveis

Em muitas linguagens de programação, é obrigatório declarar todas as variáveis no início do código, mas no JavaScript isso não é necessário. A linguagem permite que variáveis sejam declaradas em qualquer ponto do script, oferecendo maior flexibilidade para o desenvolvedor.

Outro detalhe importante é que os nomes das variáveis em JavaScript são case sensitive, ou seja, diferenciam letras maiúsculas de minúsculas. Dessa forma, variáveis como ***nome***, ***Nome*** e ***NOME*** seriam tratadas como identificadores distintos.



Javascript - Variáveis

Em JavaScript, var possui escopo de função, ou seja, uma variável declarada com var é acessível em toda a função em que foi criada, mesmo dentro de if, for ou while, e se declarada fora de qualquer função, é global. Já let possui escopo de bloco, existindo apenas dentro do bloco { } em que foi declarado, o que torna seu uso mais restrito e previsível.

```
var nome = "Ana";                    // String
let idade = 25;                      // Number
const estudante = true;              // Boolean
let frutas = ["Maçã", "Banana"]; // Array
let pessoa = { nome: "Carlos", idade: 30 }; // Objeto
let endereco = null;                 // Null
let telefone;                        // Undefined

console.log(nome, idade, estudante, frutas, pessoa, endereco, telefone);
```

Fonte: Elaboração Própria.

Javascript - Variáveis

```
console.log("=== 01 – Variáveis e Tipos ===");

const nome = "João";
let idade = 20;
const estudante = true;

console.log(nome);
console.log(idade);
console.log(estudante);

console.log(typeof nome);
console.log(typeof idade);
console.log(typeof estudante);

console.log(`Olá, ${nome}! Você tem ${idade} anos.`);
```



Tipos de dados em JavaScript

String - "Olá"

Number - 10, 3.14

Boolean - true, false

Undefined - valor não definido

Null - ausência intencional de valor

Object - { nome: "Ana", idade: 20 }

Array - [10, 20, 30]

Symbol - identificador único

BigInt - números inteiros muito grandes

typeof - verifica o tipo de um valor

*O JavaScript identifica automaticamente o tipo de dado armazenado



Operadores em JavaScript

Aritméticos

+ ⟶ soma dois valores

- ⟶ subtrai um valor de outro

* ⟶ multiplica valores

/ ⟶ divide um valor por outro

% ⟶ retorna o resto da divisão

** ⟶ realiza uma exponenciação

++ ⟶ incrementa o valor em 1

-- ⟶ decrementa o valor em 1

Comparação

== ⟶ compara os valores, permitindo conversão de tipo

=== ⟶ compara valor e tipo

!= ⟶ verifica se os valores são diferentes

!== ⟶ verifica se valor ou tipo são diferentes

> ⟶ maior que

< ⟶ menor que

>= ⟶ maior ou igual a

<= ⟶ menor ou igual a



Operadores em JavaScript

Lógicos

&& ⟶ E: todas as condições devem ser verdadeiras

|| ⟶ OU: pelo menos uma condição deve ser verdadeira

! ⟶ NÃO: inverte o resultado lógico

Atribuição

= ⟶ atribui um valor

+= ⟶ soma e atribui

-= ⟶ subtrai e atribui

*= ⟶ multiplica e atribui

/= ⟶ divide e atribui

%= ⟶ calcula o resto e atribui



Operadores em JavaScript

```
console.log("=== 02 – Operadores ===");

const nota1 = 8;
const nota2 = 7;

const media = (nota1 + nota2) / 2;

console.log("Média:", media);
console.log("Aprovado:", media >= 7);
console.log("Nota1 === Nota2:", nota1 === nota2);
```



Javascript - Condicionais

Em JavaScript, uma condicional é uma estrutura de controle que permite executar diferentes blocos de código dependendo de uma condição booleana. A mais comum é o if, que avalia uma expressão e executa o bloco de código associado se a condição for verdadeira, podendo ser complementado por else para executar outro bloco caso a condição seja falsa, e por else if para testar múltiplas condições em sequência.

```
var idade = 20;

if (idade >= 18) {
  var mensagem = "Você é maior de idade.";
} else {
  var mensagem = "Você é menor de idade.";
}

console.log(mensagem);
```

Fonte: Elaboração Própria.

Javascript - Condicionais

```
console.log("=== 03 – Condicionais ===");

const nota = 8;

if (nota >= 9) {
    console.log("Excelente");
} else if (nota >= 7) {
    console.log("Aprovado");
} else {
    console.log("Reprovado");
}

const resultado = nota >= 7 ? "Aprovado" : "Reprovado";
console.log(resultado);
```



Estruturas de repetição

- As estruturas de repetição permitem executar um bloco de código várias vezes, enquanto uma determinada condição for atendida ou durante uma sequência definida.
  - for → utilizado quando sabemos ou controlamos a quantidade de repetições.
  - while → repete enquanto uma condição for verdadeira.
  - do...while → executa o bloco pelo menos uma vez e depois verifica a condição.
  - break → interrompe a repetição imediatamente.
  - continue → interrompe a iteração atual e passa para a próxima.



Estruturas de repetição

```
console.log("=== 04 – Repetições ===");

for (let i = 1; i <= 10; i++) {
    console.log(i);
}

console.log("Pares:");

for (let i = 1; i <= 10; i++) {
    if (i % 2 === 0) {
        console.log(i);
    }
}
```



Javascript - Funções

Em JavaScript, funções são blocos de código que executam uma tarefa específica e podem retornar um valor. Elas permitem organizar o código e evitar repetição, recebendo parâmetros como entrada e devolvendo resultados com return.

Funções podem ser declaradas de forma tradicional (function nome() {}) ou como arrow functions (const nome = () => {}). Elas são usadas para manipular dados, arrays, elementos HTML e eventos, tornando o código mais modular, legível e fácil de manter.



Javascript - Funções

```
console.log("=== 05 – Funções ===");

function somar(a, b) {
    return a + b;
}

const resultado = somar(10, 20);

console.log("Resultado:", resultado);

const multiplicar = (a, b) => a * b;

console.log("Multiplicação:", multiplicar(5, 4));
```



Javascript - Exemplos de Funções

- **console.log()** – Mostra mensagens no console (útil para depuração).
- **alert() / prompt() / confirm()** – Interação básica com o usuário.
- **document.getElementById()** / document.querySelector() – Seleciona elementos do HTML.
- **.addEventListener()** – Adiciona eventos (cliques, scroll, input, etc.).
- **.innerHTML / .textContent** – Altera o conteúdo de elementos HTML.



Javascript - Arrays

- Arrays são estruturas utilizadas para armazenar vários valores em uma única variável.
- Os elementos são organizados em uma sequência e podem ser acessados por sua posição, chamada de índice.
- Exemplo:
  - const frutas = ["Maçã", "Banana", "Laranja"];



Javascript - Arrays

```
console.log("=== 06 – Arrays ===");

const notas = [7, 8, 5, 9, 10];

console.log("Notas:", notas);
console.log("Quantidade:", notas.length);

notas.forEach((nota, indice) => {
    console.log(indice, nota);
});

const aprovados = notas.filter(nota => nota >= 7);

console.log("Aprovados:", aprovados);
```



Objetos em JavaScript

- Objetos são estruturas utilizadas para armazenar informações relacionadas em propriedades e comportamentos em métodos.
  - Propriedades → armazenam dados do objeto.
  - Acesso → permite consultar ou alterar suas propriedades.
  - Métodos → funções associadas ao objeto.
  - this → referencia o próprio objeto dentro de seus métodos.



Objetos em JavaScript

```
console.log("=== 07 – Objetos ===");

const aluno = {
    nome: "Carlos",
    idade: 20,
    curso: "ADS",

    apresentar() {
        console.log(`Meu nome é ${this.nome} e curso ${this.curso}.`);
    }
};

console.log(aluno.nome);
console.log(aluno.curso);

aluno.apresentar();
```



Javascript - Exemplos de Código

```javascript
// Dark mode
const toggleBtn = document.getElementById('toggle-dark');
toggleBtn.addEventListener('click', () => {
  document.body.classList.toggle('dark');
});

// Botão voltar ao topo
const backToTop = document.getElementById('back-to-top');
window.addEventListener('scroll', () => {
  backToTop.style.display = window.scrollY > 200 ? 'block' : 'none';
});
backToTop.addEventListener('click', () => {
  window.scrollTo({ top: 0, behavior: 'smooth' });
});
```

Fonte: Elaboração Própria.

Javascript - Exemplos de Código

```javascript
// Efeito de digitação (typewriter)
const text = "Bem-vindo ao site de exemplos JavaScript!";
const typewriter = document.getElementById('typewriter');
let index = 0;
function type() {
  if (index < text.length) {
    typewriter.textContent += text.charAt(index);
    index++;
    setTimeout(type, 100);
  }
}
type();
```

Fonte: Elaboração Própria.

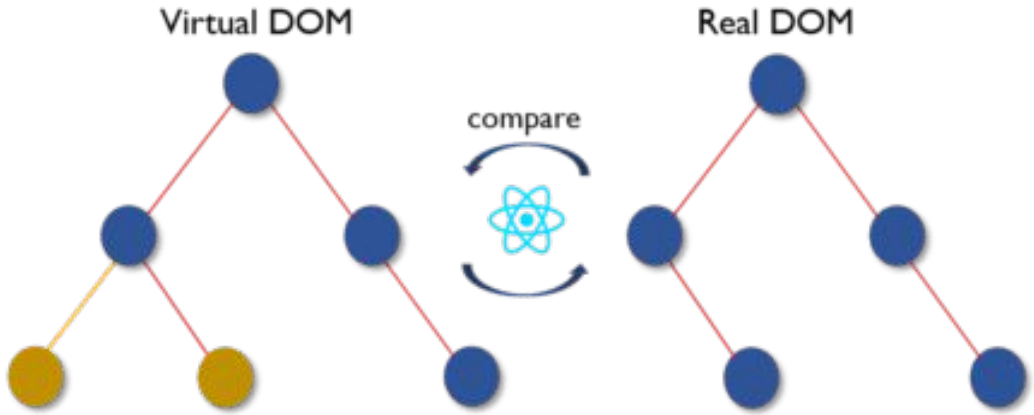
Javascript - Exemplos de Código

```
// Pop-up (modal)
const modal = document.getElementById('modal');
document.getElementById('open-modal').addEventListener('click', () => {
  modal.style.display = 'block';
});
document.getElementById('close-modal').addEventListener('click', () => {
  modal.style.display = 'none';
});
window.addEventListener('click', (e) => {
  if (e.target === modal) modal.style.display = 'none';
});
```

Fonte: Elaboração Própria.

Definição de DOM (Document Object Model)

O DOM é uma representação em árvore da estrutura de uma página web, permitindo que o JavaScript altere seu conteúdo. O Virtual DOM (usado no React) é uma cópia mais rápida desse modelo. Quando algo muda, o React primeiro atualiza essa cópia, compara com o DOM real e só aplica as diferenças, tornando tudo mais rápido e eficiente.



Fonte: https://www.mindbowser.com

Selecionando elementos do DOM

- O JavaScript permite localizar elementos HTML no DOM para posteriormente consultar ou modificar seu conteúdo.
  - getElementById() → seleciona um elemento pelo id
  - querySelector() → seleciona o primeiro elemento que corresponde a um seletor CSS
  - querySelectorAll() → seleciona todos os elementos que correspondem ao seletor CSS



Alterando o DOM com JavaScript

- O JavaScript permite modificar elementos HTML diretamente pelo DOM, alterando seu conteúdo, valores e classes.
  - textContent ⟶ altera o texto de um elemento
  - value ⟶ altera o valor de campos de formulário
  - classList ⟶ adiciona, remove ou alterna classes CSS
  - createElement() ⟶ cria novos elementos HTML
  - appendChild() ⟶ adiciona elementos ao documento



Exemplo: JS + DOM

```
// Seleciona o botão pelo ID
const botao = document.querySelector("#botao");

// Seleciona o elemento que exibirá a mensagem
const texto = document.querySelector("#mensagem");

// Adiciona um evento de clique ao botão
botao.addEventListener("click", () => {

    // Altera o texto do elemento
    texto.textContent = "Olá! O conteúdo foi alterado.";
});
```



Atividade de Revisão

Link: https://wayground.com/

Material: https://github.com/deivisontakatu/aula-javascript



Atividade

1. Crie um projeto e utilize os arquivos index.html, style.css e script.js incluindo o JS de forma Externa ao HTML, através da tag <script src>.

2. Adicione pelo menos 5 funcionalidades em JavaScript, seguindo os exemplos apresentados em aula.

3. Adicione um arquivo Readme.md no projeto, descrevendo as funcionalidades implementadas. O Arquivo deve conter pelo menos 100 linhas.



Atividade

4. Em seguida, utilize o Git no VS Code para integrar o projeto a um repositório no GitHub, inicializando o versionamento e realizando a publicação do código.

5. Realize melhorias no projeto e a adição de novos recursos, garantindo que o index.html tenha mais de 500 linhas, o style.css tenha mais de 400 linhas e o script.js tenha mais de 300 linhas.

6. Por fim, comente todo o código explicando o que cada parte faz. Documente tudo em um arquivo com o código, repositório e prints e realize o envio na plataforma CANVAS.



Referências

- CARDOSO, Márdel Vinicius de Faria. Desenvolvimento Web para o ensino superior. Rio de Janeiro: Axcel Books, 2004. 277 p. ISBN 857323220X.
- CRANE, Dave; PASCARELLO, Eric; JAMES, Darren. Ajax em ação. São Paulo: Pearson Prentice Hall, 2007. 452 p. ISBN 8576051095.
- FLANAGAN, David. JavaScript: o guia definitivo. 6. ed. Porto Alegre: Bookman, 2014. 1 recurso online. ISBN 9788565837484.
- FRAIN, Ben. Responsive web design with HTML5 and CSS3. Limited: Packt Publishing, 2012.
- HAROLD, Elliotte R. Refatorando HTML. Porto Alegre: Bookman, 2010. 1 recurso online. ISBN 9788577806706.
- LEMAY, Laura; COLBURN, Rafe; TYLER, Denise (colab.). Aprenda a criar páginas Web com HTML e XHTML em 21 dias. São Paulo: Pearson Education, 2002. 1110 p. ISBN 8534614288.
- MILETTO, Evandro Manara. Desenvolvimento de software II: introdução ao desenvolvimento Web com HTML, CSS, JavaScript e PHP. Porto Alegre: Bookman, 2014. 1 recurso online. ISBN 9788582601969.
- PINTO, Marcos José. Adobe Edge Animate CC: animação e interatividade para a Web. São Paulo: Érica, 2014. 1 recurso online. ISBN 9788536518602.
- TERUEL, Evandro Carlos. HTML5: guia prático. 2. ed. São Paulo: Érica, 2014. 1 recurso online. ISBN 9788536519296.



Facens

Web Design



JS: Sintaxe, Variáveis e Estruturas

Prof. Me. Deivison S. Takatu
Email: deivison.takatu@facens.br

NOTA MÁXIMA NO MEC 5